OOP Concepts Study Guide

1. Static Keyword

What is Static?

The `static` keyword in Java belongs to the class rather than any specific instance of the class. Static members are shared among all instances of the class.

Static Variables

Static variables are also called class variables. They are shared by all objects of the class.

```java
public class Student {
    static int totalStudents = 0;  // Static variable
    String name;

    public Student(String name) {
        this.name = name;
        totalStudents++;  // Incremented for each new student
    }
}
```

Static Methods

Static methods belong to the class and can be called without creating an object.

```java
public class MathUtils {
    public static int add(int a, int b) {
        return a + b;
    }

    public static double calculateArea(double radius) {
        return 3.14159 * radius * radius;
    }
}

// Usage
int result = MathUtils.add(5, 3);  // No object creation needed
```

Static Blocks

Static blocks are executed when the class is first loaded into memory.

```java
public class Database {
    static String connectionString;

    static {
        // Static block - runs once when class is loaded
        connectionString = "jdbc:mysql://localhost:3306/mydb";
        System.out.println("Database connection initialized");
    }
}
```

Key Rules for Static

- Static methods cannot access non-static variables or methods directly
- Static methods cannot use `this` or `super` keywords
- Static variables are initialized when the class is first loaded
- Static methods can be called using the class name

```java
public class Example {
    static int staticVar = 10;
    int instanceVar = 20;

    public static void staticMethod() {
        System.out.println(staticVar);        // ✓ Valid
        // System.out.println(instanceVar);   // ✗ Error - cannot access instance variable
        // this.instanceVar = 5;              // ✗ Error - cannot use 'this' in static context
    }
}
```

2. Interface

What is an Interface?

An interface in Java is a contract that defines what methods a class must implement. It contains abstract methods (methods without implementation) and constants.

Basic Interface Syntax

```java
public interface Animal {
    // All variables in interface are public, static, and final by default
    String KINGDOM = "Animalia";

    // All methods in interface are public and abstract by default
    void makeSound();
    void move();
}
```

Implementing an Interface

```java
public class Dog implements Animal {
    @Override
    public void makeSound() {
        System.out.println("Woof! Woof!");
    }

    @Override
    public void move() {
        System.out.println("Dog runs on four legs");
    }
}

public class Bird implements Animal {
    @Override
    public void makeSound() {
        System.out.println("Tweet! Tweet!");
    }

    @Override
    public void move() {
        System.out.println("Bird flies in the sky");
    }
}
```

Multiple Interface Implementation

A class can implement multiple interfaces.

```java
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

public class Duck implements Animal, Flyable, Swimmable {
    @Override
    public void makeSound() {
        System.out.println("Quack! Quack!");
    }

    @Override
    public void move() {
        System.out.println("Duck waddles");
    }

    @Override
    public void fly() {
        System.out.println("Duck flies short distances");
    }

    @Override
    public void swim() {
        System.out.println("Duck swims in water");
    }
}
```

Interface Inheritance

Interfaces can extend other interfaces.

```java
interface Vehicle {
    void start();
    void stop();
}

interface Car extends Vehicle {
    void honk();
    void accelerate();
}

public class Sedan implements Car {
    @Override
    public void start() {
        System.out.println("Car engine started");
    }

    @Override
    public void stop() {
        System.out.println("Car stopped");
    }

    @Override
    public void honk() {
        System.out.println("Beep! Beep!");
    }

    @Override
    public void accelerate() {
        System.out.println("Car is accelerating");
    }
}
```

Default Methods in Interfaces (Java 8+)

Interfaces can have default method implementations.

```java
interface Drawable {
    void draw();  // Abstract method

    default void display() {  // Default method
        System.out.println("Displaying the drawing");
        draw();
    }
}

public class Circle implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a circle");
    }
    // display() method is inherited and can be used as-is
}
```

3. Final Keyword

What is Final?

The `final` keyword in Java is used to restrict modification. It can be applied to variables, methods, and classes.

Final Variables

Final variables cannot be reassigned once initialized.

```java
public class Constants {
    final int MAX_SIZE = 100;           // Final instance variable
    static final double PI = 3.14159;  // Final static variable (constant)

    public void example() {
        final int localVar = 50;  // Final local variable
        // localVar = 60;         // ✗ Error - cannot reassign final variable
    }
}
```

Final Methods

Final methods cannot be overridden by subclasses.

```java
public class Parent {
    public final void importantMethod() {
        System.out.println("This method cannot be overridden");
    }

    public void normalMethod() {
        System.out.println("This method can be overridden");
    }
}

public class Child extends Parent {
    // @Override
    // public void importantMethod() {  // ✗ Error - cannot override final method
    //     System.out.println("Trying to override");
    // }

    @Override
    public void normalMethod() {  // ✓ Valid - can override non-final method
        System.out.println("Overridden method");
    }
}
```

Final Classes

Final classes cannot be extended (subclassed).

```java
public final class ImmutableClass {
    private final String value;

    public ImmutableClass(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }
}

// public class ExtendedClass extends ImmutableClass {  // ✗ Error - cannot extend final class
// }
```

Final Parameters

Method parameters can also be declared as final.

```java
public class Example {
    public void processData(final String data) {
        // data = "modified";  // ❌ Error - cannot modify final parameter
        System.out.println("Processing: " + data);
    }
}
```

Blank Final Variables

Final variables that are not initialized at declaration must be initialized in constructors.

```java
public class Student {
    private final String studentId;  // Blank final variable
    private final String name;

    public Student(String id, String name) {
        this.studentId = id;  // Must be initialized in constructor
        this.name = name;
    }

    // Getters only - no setters for final variables
    public String getStudentId() {
        return studentId;
    }

    public String getName() {
        return name;
    }
}
```

Quiz Questions

1. Which of the following statements about static methods is TRUE? a) Static methods can access instance variables directly b) Static methods can use the 'this' keyword c) Static methods can be called without creating an object of the class d) Static methods can be overridden

Correct Answer: c) Static methods can be called without creating an object of the class

2. What happens when a class implements an interface? a) The class inherits all methods from the interface with their implementations b) The class must provide implementations for all abstract methods

in the interface c) The class automatically becomes abstract d) The class cannot have its own methods

Correct Answer: b) The class must provide implementations for all abstract methods in the interface

3. Which of the following can be declared as final? a) Variables only b) Methods only c) Classes only d) Variables, methods, and classes

Correct Answer: d) Variables, methods, and classes

4. What is true about interface variables? a) They are private by default b) They can be modified after initialization c) They are public, static, and final by default d) They must be declared as abstract

Correct Answer: c) They are public, static, and final by default

5. When is a static block executed? a) Every time an object is created b) When the first static method is called c) When the class is first loaded into memory d) When the main method is executed

Correct Answer: c) When the class is first loaded into memory

---

Practice Coding Questions

1. Static Implementation Create a class called `Counter` that keeps track of how many objects have been created. Include:

- A static variable to count instances
- A constructor that increments the counter
- A static method `getCount()` that returns the current count
- An instance method `reset()` that sets the counter back to 0

2. Interface Design Design an interface called `Shape` with methods `calculateArea()` and `calculatePerimeter()`. Then create two classes `Rectangle` and `Triangle` that implement this interface. The Rectangle should take length and width as parameters, while Triangle should take three sides as parameters.

3. Final Implementation Create a final class called `BankAccount` with:

- Final instance variables for account number and account holder name
- A constructor to initialize these values
- Methods to deposit and withdraw money (balance should not be final)
- Ensure that account number and holder name cannot be changed after object creation

4. Combined Concepts Create an interface called `Printable` with a method `print()`. Then create a class `Document` that implements this interface and has:

- A static variable to count total documents created
- A final variable for document ID
- A static method to get the total document count
- Implementation of the print() method

5. Advanced Static Usage Create a utility class called `StringUtils` that:

- Cannot be instantiated (hint: use private constructor)
- Has static methods for common string operations: `reverse(String)`, `isPalindrome(String)`, and `countVowels(String)`
- Has a static final variable containing all vowels
- Make the class final to prevent inheritance